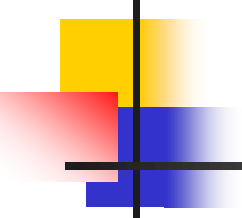


Verilog 硬體描述語言 (I)

- Verilog的模組與架構
- Verilog HDL編譯器於電路合成的應用



Four Kinds of Models

- (1) Switch Level Model (低階交換模型) or Transistor Model (電晶體層)
- (2) Gate level Model (邏輯閘層次模型)
- (3) Data Flow Model (資料處理模型) or Register Transfer Level (暫存器轉移層次)
- (4) Behavior Model (行為模型)

Format for Verilog Modules

module 模組名稱

埠列與埠的宣告 (input, output, inout ... 等)

wire, wand, wor 與 reg ... 等資料型態變數的宣告

引用較低階之模組的別名

assign 資料處理層級之描述

always 行為層級之描述區塊

begin

// 資料處理與指定 ... 等描述 或

// task 任務 與 function 函數 的使用

end

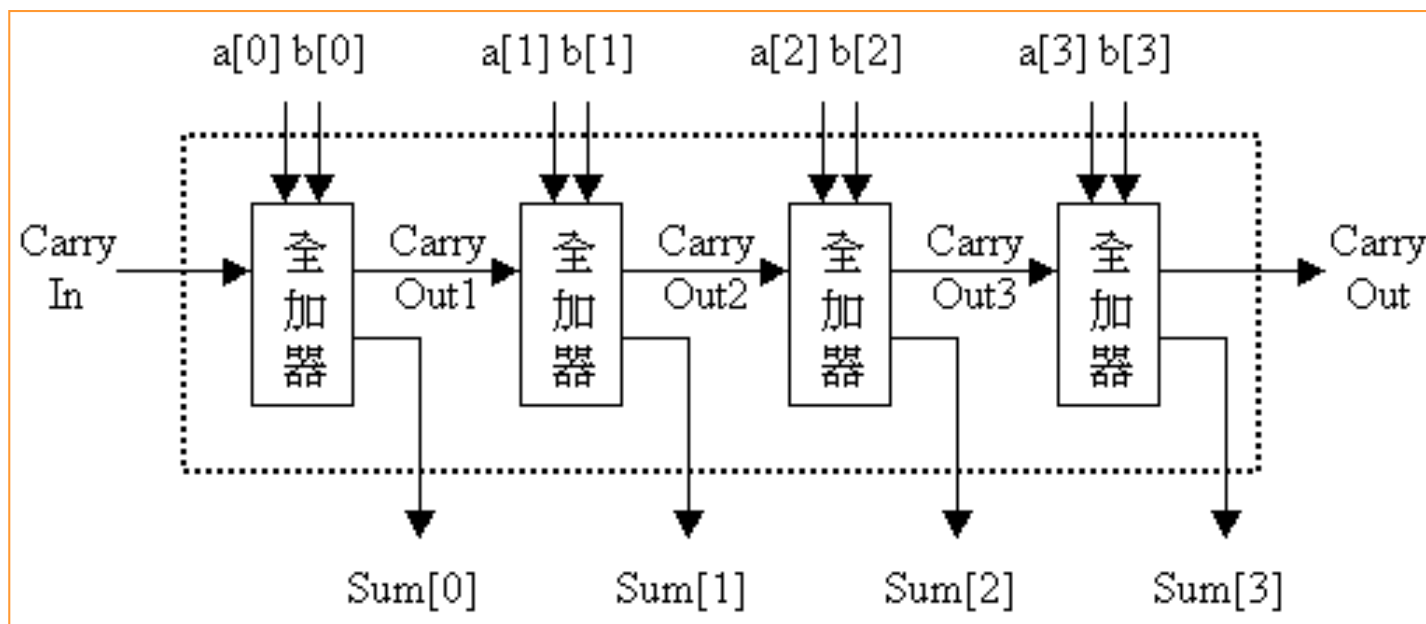
function 函數 或 (以及) task 任務的宣告

endmodule

always, for, while, case,
case, casez ... 等

模組的「別名」

- ❑ 模組如同模子，我們能用這個模子來創造出所需要的物件。
- ❑ 每個從模子中創造出來的物件都是獨立的，有其自己的名字、變數、參數與輸出入埠介面的宣告。這個從模組創造物件的過程我們稱為：**為模組取「別名」**。
- ❑ 範例：一個4個位元的漣波進位加法器 (**4 Bits Ripple Carry Adder**)，最上層的區塊擁有4個從 FullAdd 模組中創造出來的「別名」，分別為：**fa0、fa1、fa2 及 fa3**；每個別名均包含了構成 FullAdd 模組所需要的基本邏輯閘。
後一級FA需等前一級FA產生Carry



4 Bits Ripple Carry Adder

// FullAdd4.V, 4位元漣波進位計數器

```
module FullAdd4 (a, b, Carry_In, Sum, Carry_Out);
```

```
input  [3:0] a, b;
```

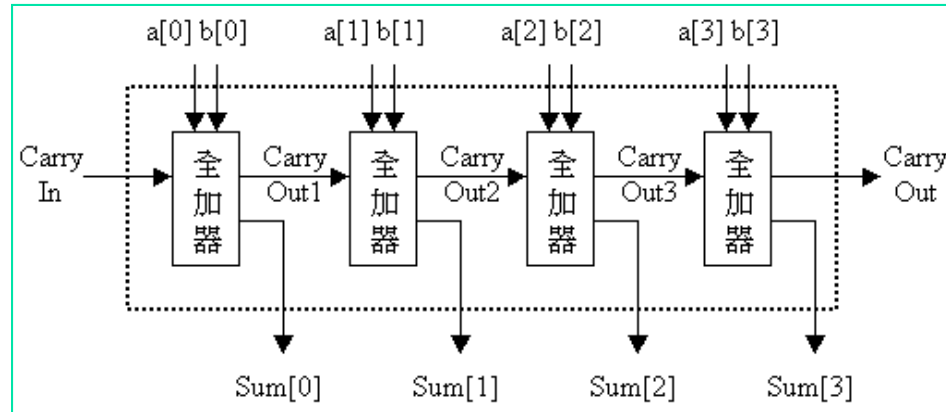
```
input  Carry_In;
```

```
output [3:0] Sum;
```

```
output Carry_Out;
```

```
wire [3:0] Sum;
```

```
wire Carry_Out;
```



```
FullAdd fa0 (a[0], b[0], Carry_In, Sum[0], Carry_Out1);
```

```
FullAdd fa1 (a[1], b[1], Carry_Out1, Sum[1], Carry_Out2);
```

```
FullAdd fa2 (a[2], b[2], Carry_Out2, Sum[2], Carry_Out3);
```

```
FullAdd fa3 (a[3], b[3], Carry_Out3, Sum[3], Carry_Out);
```

```
endmodule
```

別名

• Sub_module (defined in next page)

In order

1 Bit Full Adder

- 1位元全加器 (1 Bits Full Adder) 的邏輯符號



- 電路描述：

```
module FullAdd (a, b, CarryIn, Sum, CarryOut);
```

```
input  a, b, CarryIn;
```

```
output Sum, CarryOut;
```

```
wire Sum, CarryOut;
```

```
assign {CarryOut, Sum} = a + b + CarryIn;
```

```
endmodule
```

在 $a + b + \text{CarryIn}$ 的結果中：

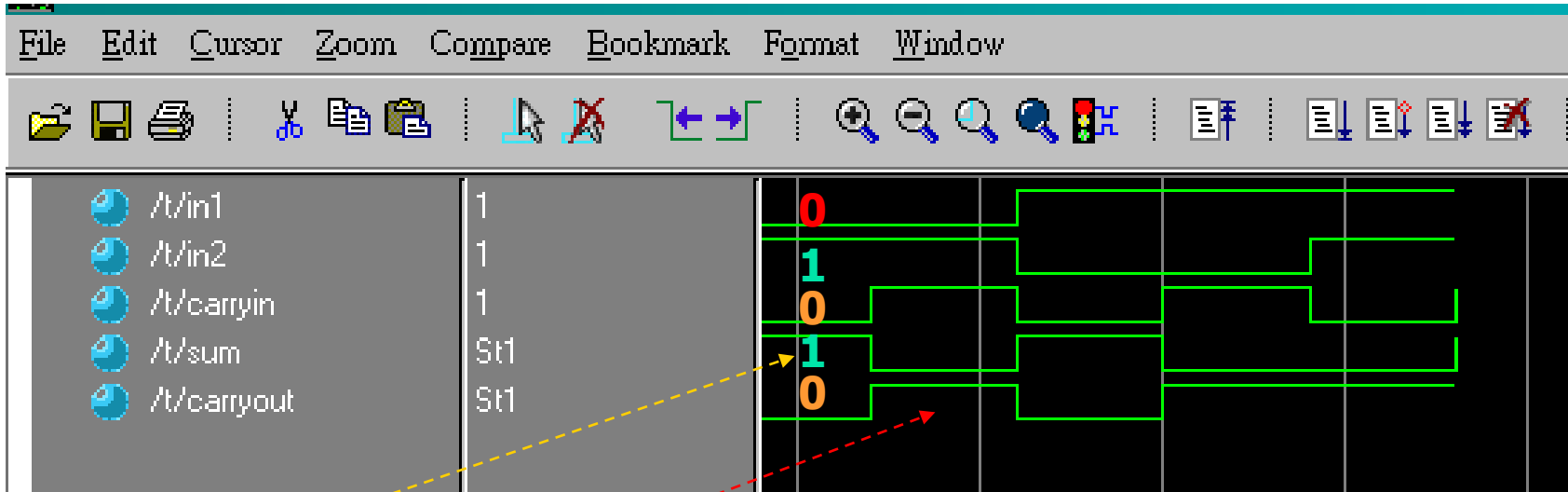
$\text{CarryOut} = \text{MSB}$

$\text{Sum} = \text{LSB}$

Max. value=11

Simulation of 1 Bit Full Adder

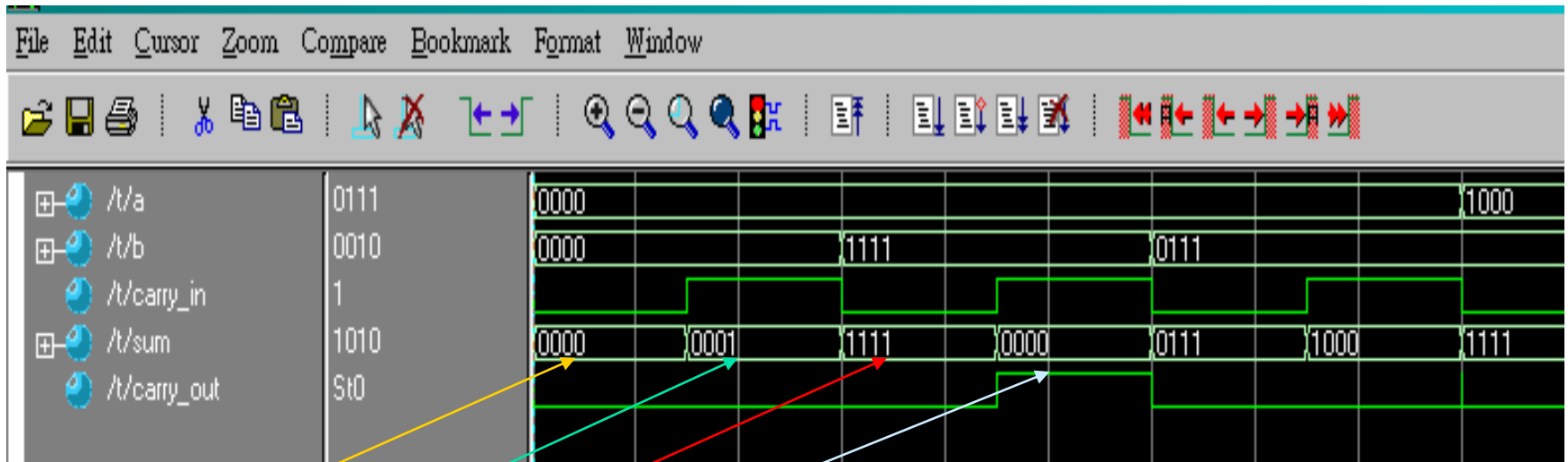
Core: 1_01_FullAdd



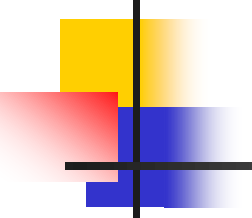
$$\begin{cases} \text{Add1} = \text{In1} + \text{In2} + \text{CarryIn} = 0 + 1 + 0 = 1 \Rightarrow \text{Carry} = 0, \text{Sum} = 1 \\ \text{Add2} = \text{In1} + \text{In2} + \text{CarryIn} = 0 + 1 + 1 = 10 \Rightarrow \text{Carry} = 1, \text{Sum} = 0 \end{cases}$$

Simulation of 4 Bits Ripple Carry Adder

Core:1_02_FullAdd4



$$\begin{cases} \text{Add1} = a + b + \text{carry_in} = 0000 + 0000 + 0 = 0000 \Rightarrow \text{Carry} = 0, \text{Sum} = 0000 \\ \text{Add2} = a + b + \text{carry_in} = 0000 + 0000 + 1 = 0001 \Rightarrow \text{Carry} = 0, \text{Sum} = 0001 \\ \text{Add3} = a + b + \text{carry_in} = 0000 + 1111 + 0 = 1111 \Rightarrow \text{Carry} = 0, \text{Sum} = 1111 \\ \text{Add4} = a + b + \text{carry_in} = 0000 + 1111 + 1 = 10000 \Rightarrow \text{Carry} = 1, \text{Sum} = 0000 \end{cases}$$



Syntax for Verilog

- ❑ Verilog 是由一連串的標記 (token) 所組成，這些標記可以是：空白 (Whitespace)、註解 (Comments)、運算子 (Operators)、數值 (Numbers)、識別字 (Identifiers) 與關鍵字 (Keywords) 以及底線 (Underscore characters) 和問號 (Question Marks)。
- ❑ 識別字 (Identifiers) 的大小寫是不同的，例：Even 和 even 是二個不同的標記名稱。
- ❑ 所有的關鍵字 (Keywords) 名稱必須使用小寫來表示。
- ❑ 空白 (Whitespace)：Verilog 對空白的定義為：只要是空格 (Blank Space, ' ')、跳格 (Tab, ' \t') 和換行符號 (New Line, ' \n') 皆屬於空白字元。

Comments

❑ **註解 (Comments)** : 可以在 Verilog 電路描述檔中，特定加入對此模組的說明，便於日後再閱讀或稱修改此模組時、可以很快地了解這個模組的功能及模組中特定描述的說明資料，這個說明的部份就稱之為註解。

❑ **註解的寫法**，有下列二種：

➤ 以 “//”為開頭的註解寫法，稱為「單行註解」(One Line Comment)。

✓ 例：// FullAdd4.V, 4位元漣波進位計數器

wire a = b & c; // 宣告1條接線 a, 並指定它為 b & c

➤ 以 “/*”為開頭、並以 “*/”當作結尾的註解寫法，稱為「多行註解」(Multiple Line Comment)，惟多行註解內不可以再有多行註解。

✓ 例：

/* 若 op = ADD, 則 Result = a + b; 若 op = SUB, 則 Result = a + b;
若 op = AND, 則 Result = a & b; 若 op = OR, 則 Result = a | b;
若 op 不為上述4種之1的話, 則 Result = a; */

/* 錯誤的多行註寫法

/* 因為多行註解內有另一個多行註解 */ */

Operators

□ 運算子 (Operators): 運算子有下列三種格式。

➤ 單元 (Unary) 運算子

✓ 單元運算子($\sim$)必須放在單一運算元的前面。

例：assign a = $\sim$ b; // $\sim$, Bit-wise NOT

➤ 二元 (Binary) 運算子

✓ 二元運算子放在二個運算元之間。

例：assign a = b & c; // &, Bit-wise AND

➤ 三元 (Ternary) 運算子

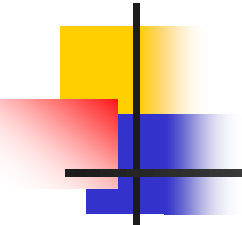
✓ 三元運算子就是條件運算子 (Conditional operators)、(? :)。

例：BNF 表示法：Value = Test _Condition ? True_Part : False_Part;

說明：如果 Test Condition 的值為 1 (True, 真) 的話，則 Value 的值將被設定為 True_Part 的值，否則 Value 的值將被設定為 False_Part 的值。

例：assign Out = Select ? a : b;

// 如果 Select = 1，則 Out = a，否則 Out = b。



Numbers (1)

□ **數值 (Numbers)** :數值規格分為：定長度 (Sized) 數字，以及不定長度 (UnSized) 數字等二種。

➤ **定長度 (Sized) 數字**

- ✓ 定長度數字是以 `<size>'<base format><number>` 來表示。其中 `<size>` 是以十進位來表示數字的位元數 (bits)，`<base format>` 用來定義此數值為：十六進位 ('h 或 'H)、十進位 ('d 或 'D)、八進位 ('o 或 'O)，或者是二進位 ('b 或 'B)。
- ✓ `<number>`：十六進位 ('h 或 'H)可以用的數字為 0～9 及 A～F (A、B、C、D、E 及 F)，其中 A～F 也可以用小寫的 a～f (a、b、c、d、e 及 f) 來表示。

Numbers (2)

```
input [2:0] in;
```

```
wire [15:0] Out1, [9:0] Out2, [8:0] Out3, [7:0] Out4;
```

```
// 16 Bits 的十六進位數值，左移 in 次
```

左移一次相當於乘2

```
assign Out1 = 16'h89Ab << in;
```

定長度數字是以 <size,位元數>'<base format,類型><number,數值> 來表示

```
// 10 Bits 的十進位數值，左移 in 次
```

```
assign Out2 = 10'd128 << in;
```

```
// 9 Bits 的八進位數值，左移 in 次
```

```
assign Out3 = 9'o377 << in;
```

```
// 8 Bits 的二進位數值，左移 in 次
```

```
assign Out4 = 8'b10101010 << in;
```

Numbers (3)

➤ 不定長度 (UnSized) 數字

不定長度數字 不必使用 <size> 來規定數字之位元數大小，而是使用 HDL 編譯器內定的長度。其中若沒有寫明 <base format> 的部份，則此數值內定為十進制。

例：

(不定長度)

← 一般內定為 32 bits

assign Out1 = 'h89Ab << in; // 十六進位數值，左移 in 次

assign Out2 = 128 << in; // 十進位數值，左移 in 次

assign Out3 = 'o377 << in; // 八進位數值，左移 in 次

assign Out4 = 'b10101010 << in; // 二進位數值，左移 in 次

8'-3c // 錯誤語法

-8'd79 // 定義十進制 8 位元長度數值 79 之 2 的補數

8'h3z // 8 位元長度之十六進制且最低 4 位元為高阻抗之數值

12'oz732 // 12 位元長度之八進制且最高三個位元為高阻抗數值

Strings

□ 字串 (Strings)

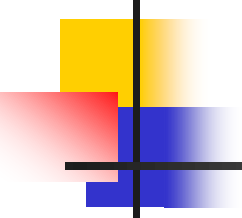
- 字串以雙引號("...")框住之一連串字元，即以一連串之8位元之ASCII碼來表示字串。
- 字串所使用之前後雙引號，僅能出現在同一行。
- 欲使用變數去儲存字串時，必須宣告一個夠大的暫存器來容納該字串所有字元。

範例：

```
reg [8*18-1:0] stringvar; //寬度為144位元(18個字(含空白)之暫存器
initial
begin
    stringvar = "Hello Verilog HDL!";
end
```

範例：

```
" Multiple lines for
    a string is not illegal"           //錯誤字串
```



Identifiers and Keywords

❑ 識別字 (Identifiers) 與關鍵字 (Keywords)

- 識別字是在 Verilog 電路描述中所給予物件的名稱。
- 識別字的第一個字元必須為字母，第二個之後的字元可為字母、數字、底線 “_” 或者是錢號 “\$” 所組成。
- 識別字大小寫是有分別的。

例：Even 和 even 是二個不同的標記名稱。

❑ 關鍵字是一組特殊的定義名稱，其功用是為了定義 Verilog 電路描述的架構。

- 所有的關鍵字 (Keywords) 名稱必須使用小寫來表示。

例：
`input CarryIn;` // input 是關鍵字, CarryIn 是識別字
`wire CarryOut;` // wire 是關鍵字, CarryOut 是識別字

Underscores and Questions

□ 底線 (Underscore characters) 和 問號 (Question Marks)

- 底線 “_”的目的在於增加模組名稱、變數名稱、function 的名稱、task 的名稱或關鍵字 ... 等等的可讀性。
- 必需注意的是上述這些名稱或關鍵字的第一個字元不能是底線。
- 問號 “?”、符號 “z”，以及底線符號 “_”也同樣具有增加數值的可讀性之作用。

例： // 12'b1010_1111_0101 等於 12'b101011110101
assign Out1 = 12'b1010_1111_0101 << in;

（圖中註釋：兩個底線符號上方標有“空白”，並有粉紅色圈標出）

例： // 8'b11??11?? 等於 8'b11zz11zz
assign Out2 = 8'b11??11?? << in;

（圖中註釋：問號上方標有“?”，並有粉紅色圈標出）

(?為任意值 (含 0, 1, z, x) , , 而z為浮接(高阻抗), x為unknown)

“Display” and “Write” Descriptions (1/2)

□ 顯示 (Display) 和 寫出 (Write)

- **\$display** 會在模擬過程中用來顯示”字串”、”表示式”或是”變數”的數值。

例：

```
$display("Hello Prof. Lin");           //顯示Hello Prof. Lin
$display($time);                        //顯示目前的時間
data1=4'b0101;
$display("The data is %b", data1);      //顯示The data is 0101
```

- **\$write** 除了無法在輸出結尾換行以外，其餘均與\$display相同。

例：

```
$write(4'b1111);
$write(" ", 5b'10101);
$writeh(" ", 5b'01111, "\n");
顯示結果為： 15 21 0F
```

“Display” and “Write” Descriptions (2/2)

Description	Default Format
\$display	十進制
\$displayb	二進制
\$displayh	十六進制
\$displayo	八進制
\$write	十進制
\$writeb	二進制
\$writeh	十六進制
\$writeo	八進制

Format	Data Display
%d or %D	十進制
%b or %B	二進制
%h or %H	十六進制
%c or %C	ASCII字元
%o or %O	八進制
%m or %M	階層名字(不需引數)
%s or %S	字串
%t or %T	目前時間格式
%e or %E	科學格式顯示實數
%f or %F	十六進制格式顯示實數
%g or %G	十進制/科學格式之短實數
%v or %V	電壓強度

Simulation Monitor

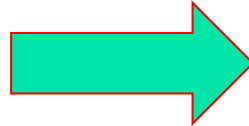
❑ 連續監視 (Continuous monitoring)

- \$monitor的格式與\$display完全相同;惟\$monitor持續監視表列中的變數值及特定訊號的變化，而\$display僅被呼叫一次。
- 監視功能可由\$monitoron啟動，而由\$monitoroff關閉。

範例：

```
module monitest;
  integer s, t;
  initial begin
    s=4;
    t=6;
    forever begin
      #5 s=t+s;
      #5 t=s-1;
    end // forever begin
  end // initial begin
  initial #40 $finish;
  initial begin
    $monitor($time, "s=%d, t=%d," s,t);
  end //initial begin
endmodule
```

執行結果



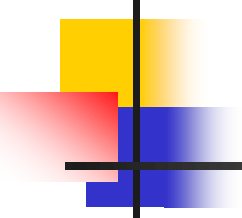
0	S=4,	t=6
5	S=10,	t=6
10	S=10,	t=9
15	S=19,	t=9
20	S=19,	t=18
25	S=37,	t=18
30	S=37,	t=36
35	S=73,	t=36

❑ 閃控監視 (Strobed monitoring)

- \$strobe可在選定的時間之下，顯示模擬資料。

範例：

```
forever @(posedge clk)
  $strobe("Time = %d, data=%f", $time, data);
//在每一時脈clk正緣時，$strobe將時間及資料訊息寫入標準的輸出檔(log file)或螢幕。
```



Stop Simulation

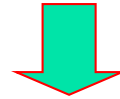
□ 結束模擬

- **\$stop** 會使模擬器保持Standby狀態，並將Verilog設定為交談模式。
- **\$finish** 會離開模擬且將控制權移轉給系統。

範例：

```
initial
begin
    clock = 1'b0;
    reset = 1'b0;
    #300 $stop;           //模擬standby，並設定為交談模式
    #600 $finish;         //結束模擬
end
```

- ❑ 埠對應 (Port Mapping) 連接的方式：將一個模組的埠與外部訊號連接的方法有兩種：
 - 依照定義模組時埠列的「順序」(in order) 來連接
 - 依「指定名稱」(by name) 的方法來連接。
 - 這兩種方法必需分開使用、不可以同時在同一個引用模組的別名中使用。



```
// 依「指定名稱」(by name) 的方法來連接, Port 的順序就無所謂了
Mux      Mux_1 (.Sel(Sel), .a(a), .b(b), .out(Mux_Out));

// 依「指定名稱」(by name) 的方法來連接, Port 的順序就無所謂了
Register8 Register8_1 (.Clock(Clock), .Reset(Reset),
                      .data(Mux_Out), .q(Reg_Out));

// 依照定義模組時埠列的「順序」(in order) 來連接
Rotate_Data Rotate_Data_1 (Clock, Reset, Left_Rotate, Reg_Out, out);
```

階層式設計範例 (1)

// Hierar.V：階層式設計範例

// 首先定義較小的模組：Mux, Register8 及
Rotate_Data 之後再去引用它們

// Multiplexor 多工器

```
module Mux (Sel, a, b, out);
```

```
    input    Sel;
```

```
    input    [7:0] a, b;
```

```
    output   [7:0] out;
```

```
    wire     [7:0] out;
```

```
    assign out = Sel ? a : b;
```

```
endmodule
```



Sel = 1 , out = a ;

Sel = 0 , out = b ;

階層式設計範例 (2)

// 8個位元的暫存器

```
module Register8 (Clock, Reset, data, q);
```

```
input  Clock, Reset;
```

```
input  [7:0] data;
```

```
output [7:0] q;
```

```
reg    [7:0] q;
```

```
always @(posedge Clock)
```

```
begin
```

```
    if (Reset)
```

```
        q = 0;
```

```
    else
```

```
        q = data;
```

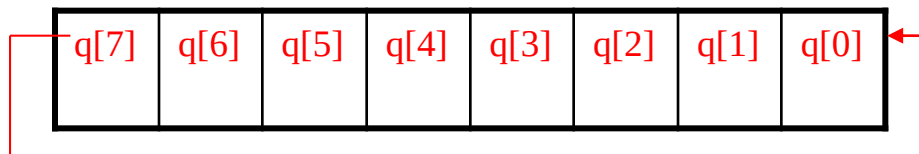
```
    end
```

```
endmodule
```

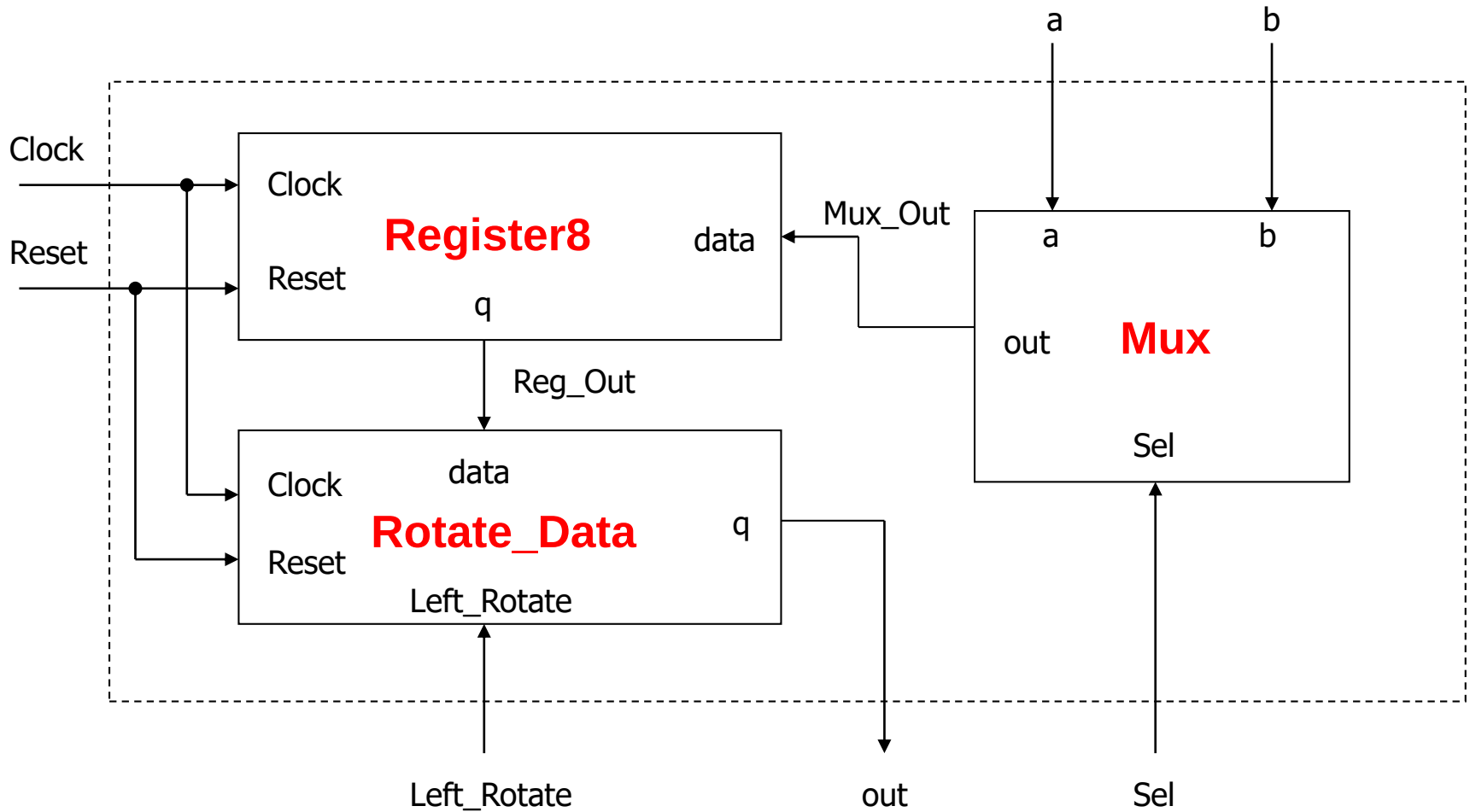


階層式設計範例 (3)

```
// 「載入」資料或「旋轉」資料
module Rotate_Data (Clock, Reset, Left_Rotate, data,
    q);
input    Clock, Reset, Left_Rotate;
input    [7:0] data;
output   [7:0] q;
reg      [7:0] q;
always @(posedge Clock)
begin
    if (Reset) // 重置
        q = 8'b0; // 將 q 清除為 0
    else
        if (Left_Rotate) // 「旋轉」資料
            q = {q[6:0], q[7]};
        else
            q = data; // 從輸入埠 data, 「載入」資料至 q
    end
end
endmodule
```

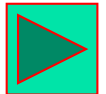


Mapping Block Diagram



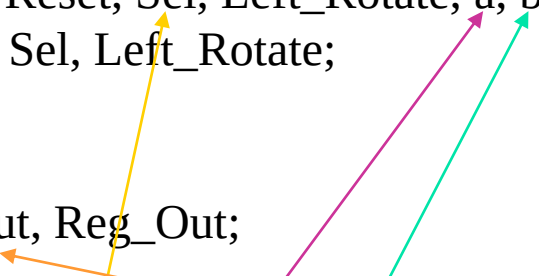
Port Mapping by Order

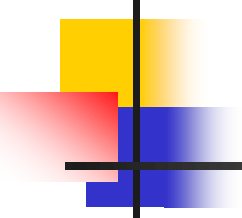
```
// -----  
// 模組 Top1：依照定義模組時埠列的「順序」(in order) 來連接  
// -----  
module Top1 (Clock, Reset, Sel, Left_Rotate, a, b, out);  
input    Clock, Reset, Sel, Left_Rotate;  
input    [7:0] a, b;  
output   [7:0] out;  
wire     [7:0] Mux_Out, Reg_Out;  
  
    Mux          Mux_1 (Sel, a, b, Mux_Out);  
    Register8     Register8_1 (Clock, Reset, Mux_Out, Reg_Out);  
    Rotate_Data  Rotate_Data_1 (Clock, Reset, Left_Rotate, Reg_Out,  
                                out);  
  
endmodule
```



Port Mapping by Name

```
// -----  
// 模組 Top2：依照「指定名稱」(by name) 的方法來連接  
// -----  
module Top2 (Clock, Reset, Sel, Left_Rotate, a, b, out);  
input  Clock, Reset, Sel, Left_Rotate;  
input  [7:0] a, b;  
output [7:0] out;  
wire   [7:0] Mux_Out, Reg_Out;  
  
// 依「指定名稱」(by name) 的方法來連接, Port 的順序就無所謂了  
Mux      Mux_1 (.Sel(Sel), .a(a), .b(b), .out(Mux_Out));  
  
// 依「指定名稱」(by name) 的方法來連接, Port 的順序就無所謂了  
Register8 Register8_1 (.Clock(Clock), .Reset(Reset),  
                      .data(Mux_Out), .q(Reg_Out));  
  
// 依照定義模組時埠列的「順序」(in order) 來連接  
Rotate_Data Rotate_Data_1 (Clock, Reset, Left_Rotate, Reg_Out,  
                           out);  
  
endmodule
```





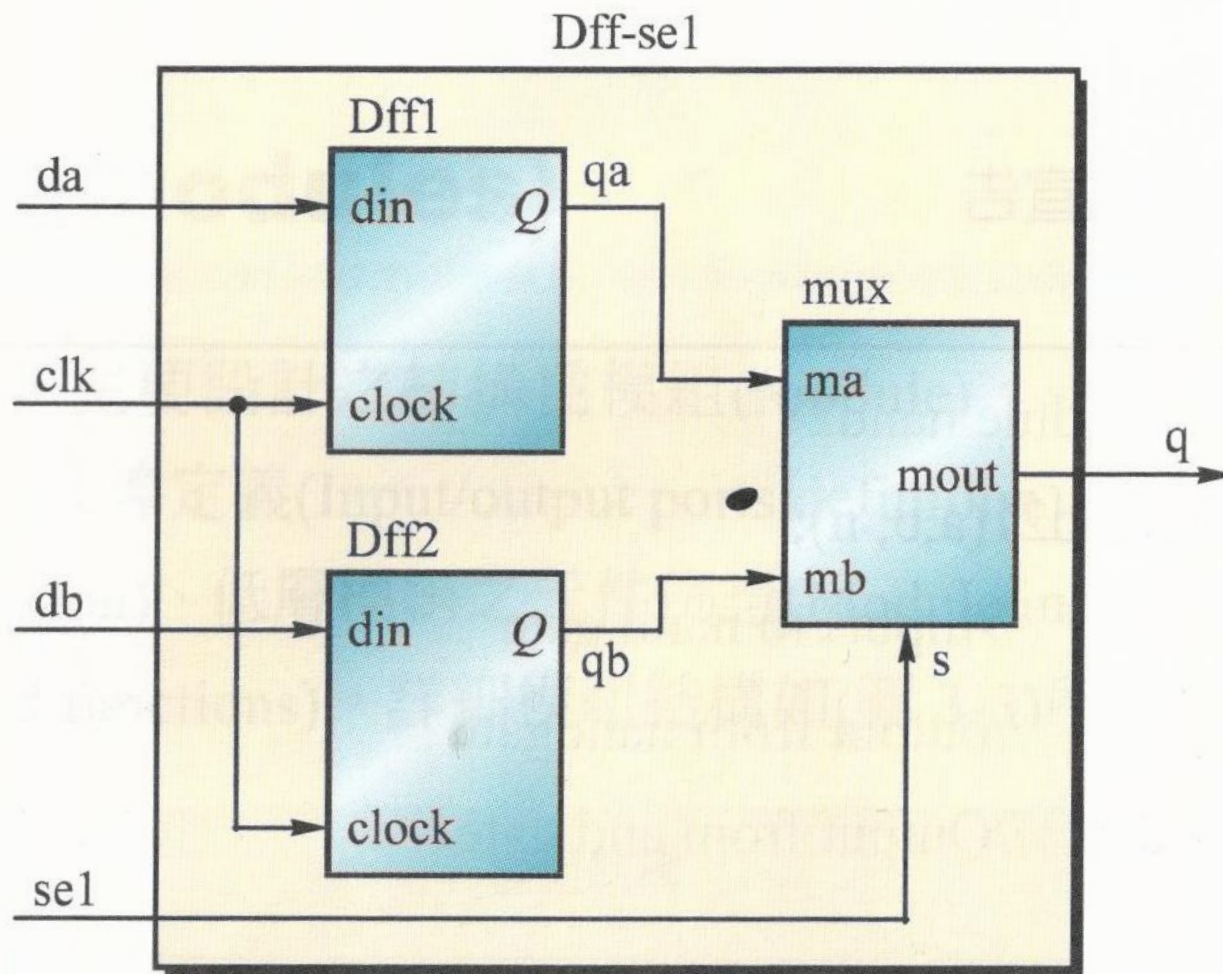
範例練習

D型正反器+多工器

(3-001)

Code: 3_001_DFF

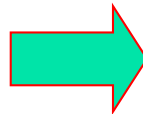
正反器選擇電路方塊圖



//filename dff.v

```
module dff(din, clk, q);  
    input din, clk;  
    output q;  
    reg q;  
    always @(posedge clk)  
        q = din;  
endmodule
```

Test bench



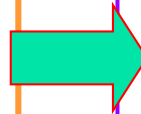
//filename test bench for dff.v

```
module dff_tb();  
    reg din, clk;  
    wire q;  
    dff top(.din(din), .clk(clk), .q(q));  
    initial  
        begin  
            #0 din<=0;  
            clk<=0;  
        end  
    always  
        begin  
            #5 clk <= !clk;  
        end  
    always  
        begin  
            #10 din <= !din;  
        end  
endmodule
```

```
//filename: dff-sel.v
```

```
module dff_sel(da, db, sel, clk, q);
    input da, db, sel, clk;
    output q;
    wire qa, qb;
    dff dff1(da, clk, qa);
    dff dff2(db, clk, qb);
    mux2_1
        mux(.ma(qa), .mb(qb), .s(sel), .mout(q));
endmodule
```

Test bench

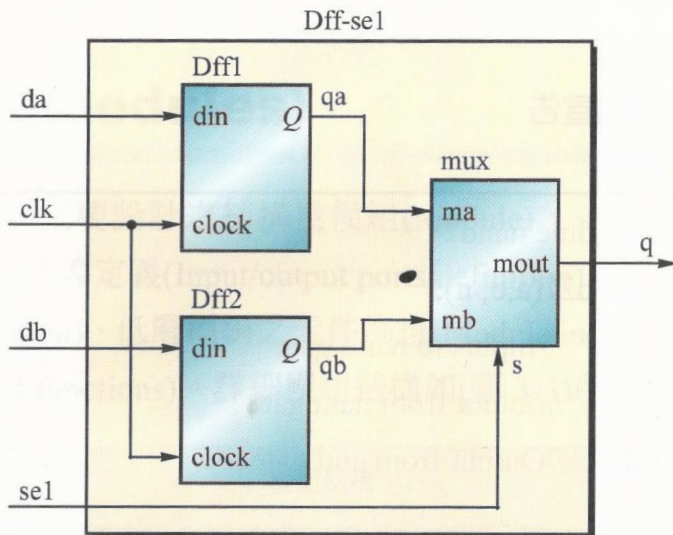


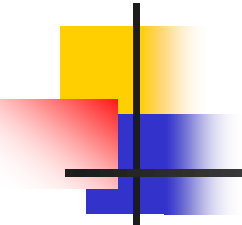
```
//filename test bench for dff-sel_tb.v
module dff_sel_tb();
    reg da, db, sel, clk;
    wire q;
    dff_sel dff(.da(da), .db(db), .sel(sel), .clk(clk), .q(q));
    initial
        begin
            #0 da<=0;
            db<=0;
            sel<=0;
            clk<=0;
        end

    always
        begin
            #1 clk <= !clk;
        end

    always
        begin
            #10 da = 0;
            db = 0;
            sel = 0;
            ...
        end

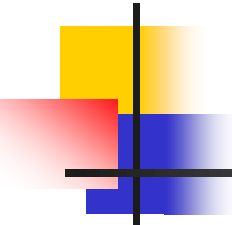
    initial #1000 $finish; // END simulation .
endmodule
```





Verilog 硬體描述語言 (I)

- Verilog的模組與架構
- Verilog HDL編譯器於電路合成的應用



Invalid Descriptions in Verilog HDL

❑ 不能用於電路合成的「敘述」

- delay 敘述
- initial 敘述
- repeat 敘述
- forever 敘述
- wait 敘述
- fork 敘述 / join 敘述
- event 敘述
- time 敘述
- deassign 敘述
- force 敘述 / release 敘述
- primitive 敘述(少用)
- case identity, not identity operations 敘述(少用)
- rtran, tranif0, tranif1, rtranif0, rtranif1 敘述(少用)

Invalid Operators in Verilog HDL

❑ === (連續三個 = 號) 運算子

事件上的相等，即0、1、
x、z都相等

❑ !== (一個 ! 號接著二個 = 號) 運算子

❑ 除法運算子 (/ , division)

❑ 取餘數運算子 (% , modulus)

例：在下面的這個例子中的述是不能由 Synopsys 來作電路合成的，因為 Synopsys 不提供除法 (/) 及 取餘數 (%) 的運算。

~~wire [7:0] a, b, c, d;~~

~~assign a = c / d;~~

~~assign b = c % d;~~

Invalid Logic Gates in Verilog HDL

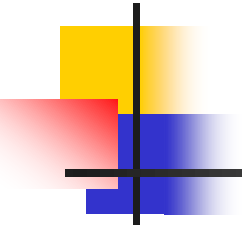
❑ 不能用於電路合成的「邏輯閘型態」：

- tranif1, tranif0, rtran, rtranif1, rtranif0 等邏輯閘型態
- triand, trior, tri1, tri0, trireg 等接線型態
- nmos, pmos, cmos, rnmos, rpmos, rcmos (少用)等元件
- pullup, pulldown 等訊號改變元件

電晶體階層的 keyword

❑ 不能在模組內有另一個階層式的名稱：

- 例如：另一個模組 (module)
- `ifdef, `endif, `else 等條件式編譯命令 (Compiler Directives)



Valid Descriptions

□ 能用於電路合成的「敘述」：

- include 敘述
- `define 敘述
- module、endmodule 敘述
- parameter 敘述
- input、output、inout 敘述
- wire、wand、wor 敘述
- reg 敘述
- begin、end 敘述
- always 敘述
- assign 敘述
- @、posedge、negedge 敘述
- for 敘述
- if、else 敘述
- case、casex、casez 敘述
- function 敘述
- task 敘述
- defparam、# 敘述

Valid Operators - Binary-wise operators (1)

□ 二元逐位元運算子 (Binary Bit-wise operators) :

- 二元逐位元運算子，包括：(1) $\sim$, Bit-wise NOT、(2) $\&$, Bit-wise AND、(3) $|$, Bit-wise OR、(4) $\wedge$, Bit-wise XOR，以及 (5) $\sim\wedge$ 或 $\wedge\sim$, Bit-wise XNOR 等5種。
- (1). $\sim$, Bit-wise NOT 逐位元運算子

$\sim$, Bit-wise NOT

b	0	1
$\sim b$	1	0

真值表

對於特定一筆輸入資料的任何一個位元：若第 n 個位置為 0、則運算結果的第 n 個位置為 1，若第 n 個位置為 1、則運算結果的第 n 個位置為 0。

➤ 例：assign $a = \sim b$;

若 $b = 4'b0010$

則 $a = 4'b1101$

Valid Operators - Binary-wise operators (2)

□ 二元逐位元運算子 (Binary Bit-wise operators) :

➤ (2). &, Bit-wise AND 逐位元運算子

&, Bit-wise AND

b	0	0	1	1
c	0	1	0	1
b & c	0	0	0	1

真值表

對於特定二筆輸入資料的任何一個位元：若第 n 個位置只要有一個為 0、則運算結果的第 n 個位置為 0，若第 n 個位置皆為 1、則運算結果的第 n 個位置為 1。

➤ 例：assign $a = b \& c;$

若 $b = 4'b0011$

$\& c = 4'b1010$

則 $a = 4'b0010$

Valid Operators - Binary-wise operators (3)

□ 二元逐位元運算子 (Binary Bit-wise operators) :

➤ (3). |, Bit-wise OR 逐位元運算子

|, Bit-wise OR

b	0	0	1	1
c	0	1	0	1
b c	0	1	1	1

真值表

對於特定二筆輸入資料的任何一個位元：若第 n 個位置只要有一個為 1、則運算結果的第 n 個位置為 1，若第 n 個位置皆為 0、則運算結果的第 n 個位置為 0。

➤ 例：assign a = b | c;

若 b = 4'b0011

| c = 4'b1010

則 a = 4'b1011

Valid Operators - Binary-wise operators (4)

□ 二元逐位元運算子 (Binary Bit-wise operators) :

➤ (4). $\wedge$, Bit-wise XOR 逐位元運算子

$\wedge$, Bit-wise XOR

b	0	0	1	1
c	0	1	0	1
$b \wedge c$	0	1	1	0

真值表

對於特定二筆輸入資料的任何一個位元：若第 n 個位置只要有一個為 1 另一個為 0、則運算結果的第 n 個位置為 1，若第 n 個位置皆為 1 或 0、則運算結果的第 n 個位置為 0。

Bit-wise XOR 和 Bit-wise XNOR 有互補的關係。

➤ 例：assign $a = b \wedge c$;

若 $b = 4'b0011$

$\wedge c = 4'b1010$

則 $a = 4'b1001$

以1,1來判斷：

➤ XOR: 相同為0，不同為1

➤ XNOR: 相同為1，不同為0

Valid Operators - Binary-wise operators (5)

□ 二元逐位元運算子 (Binary Bit-wise operators) :

➤ (5). $\sim^{\wedge}$ 或 $\wedge^{\sim}$, Bit-wise XNOR 逐位元運算子

$\sim^{\wedge}$ 或 $\wedge^{\sim}$, Bit-wise XNOR

b	0	0	1	1
c	0	1	0	1
$b \sim^{\wedge} c$	1	0	0	1

真值表

對於特定二筆輸入資料的任何一個位元：若第 n 個位置只要有一個為 1 另一個為 0、則運算結果的第 n 個位置為 0，若第 n 個位置皆為 1 或 0、則運算結果的第 n 個位置為 1。

Bit-wise XNOR 和 Bit-wise XOR 有互補的關係。

➤ 例：assign $a = b \sim^{\wedge} c$;

若 $b = 4'b0011$

$\sim^{\wedge} c = 4'b1010$

則 $a = 4'b0110$

以1,1來判斷：

➤ XOR: 相同為0，不同為1

➤ XNOR: 相同為1，不同為0

Valid Operators - Unary reduction operators (1)

□ 單元運算子 (Unary reduction operators)

單元運算子包括有：(1). $\&$, reduction AND、(2). $\sim\&$, reduction NAND、(3). $|$, reduction OR、(4). $\sim|$, reduction NOR、(5). $\wedge$, reduction XOR，以及 (6). $\sim\wedge$ 或 $\wedge\sim$, reduction XNOR 等6種。
(Binary Bit-wise operators)：

➤ (1). $\&$, reduction AND 單元運算子

$\&$, reduction AND

b	0	1	00	01	10	11
$\&b$	0	1	0	0	0	1

真值表

對於特定一筆輸入資料的任何一個位元：若第 n 個位置只要有一個為 0，則運算結果為 0，若第 n 個位置皆為 1，則運算結果為 1。

reduction AND 和 reduction NAND 有互補的關係。

➤ 例：assign $a = \&b$;

若 $b = 4'b0000$ ，則 $a = 1'b0$

若 $b = 4'b0111$ ，則 $a = 1'b0$

若 $b = 4'b1111$ ，則 $a = 1'b1$

Valid Operators - Unary reduction operators (2)

□ 單元運算子 (Unary reduction operators)

➤ (2). $\sim\&$, reduction NAND 單元運算子

$\sim\&$, reduction NAND

b	0	1	00	01	10	11
$\sim\&b$	1	0	1	1	1	0

真值表

對於特定一筆輸入資料的任何一個位元：若第 n 個位置只要有一個為 0，則運算結果為 1，若第 n 個位置皆為 1，則運算結果為 0。

reduction NAND 和 reduction AND 有互補的關係。

➤ 例：assign $a = \sim\& b$;

若 $b = 4'b0000$ ，則 $a = 1'b1$

若 $b = 4'b0111$ ，則 $a = 1'b1$

若 $b = 4'b1111$ ，則 $a = 1'b0$

Valid Operators - Unary reduction operators (3)

□ 單元運算子 (Unary reduction operators)

➤ (3). |, reduction OR 單元運算子

|, reduction OR

b	0	1	00	01	10	11
b	0	1	0	1	1	1

真值表

對於特定一筆輸入資料的任何一個位元：若第 n 個位置只要有一個為 1、則運算結果為 1，若第 n 個位置皆為 0、則運算結果為 0。

reduction OR 和 reduction NOR 有互補的關係。

➤ 例：assign a = | b;

若 $b = 4'b0000$ ，則 $a = 1'b0$

若 $b = 4'b0111$ ，則 $a = 1'b1$

若 $b = 4'b1111$ ，則 $a = 1'b1$

Valid Operators - Unary reduction operators (4)

□ 單元運算子 (Unary reduction operators)

➤ (4). $\sim|$, reduction NOR 單元運算子

$\sim|$, reduction NOR

b	0	1	00	01	10	11
$\sim b$	1	0	1	0	0	0

真值表

對於特定一筆輸入資料的任何一個位元：若第 n 個位置只要有一個為 1、則運算結果為 0，若第 n 個位置皆為 0、則運算結果為 1。

reduction NOR 和 reduction OR 有互補的關係。

➤ 例：assign $a = \sim| b$;

若 $b = 4'b0000$ ，則 $a = 1'b1$

若 $b = 4'b0111$ ，則 $a = 1'b0$

若 $b = 4'b1111$ ，則 $a = 1'b0$

Valid Operators - Unary reduction operators (5)

□ 單元運算子 (Unary reduction operators)

➤ (5). $\wedge$, reduction XOR單元運算子 → 用於奇同位產生器(奇數個1)

$\wedge$, reduction XOR

b	0	1	00	01	10	11
$\wedge b$	0	0	0	1	1	0

真值表

若輸入資料的位元數為 1 個、則運算的結果為 0，相當於是清除的動作。

若輸入資料的位元數為 2 個 (含) 以上、且輸入資料值含有奇數個 1，則運算的結果為數 1、否則必為 0。

通常用於算術邏輯運算單元 (ALU) 中的 Odd (奇同位元) 判斷旗標 (flags) 訊號。

reduction XOR 和 reduction XNOR 有互補的關係。

➤ 例：assign $a = \wedge b$;

若 $b = 4'b0000$ ，則 $a = 1'b0$

若 $b = 4'b0111$ ，則 $a = 1'b1$

若 $b = 4'b1111$ ，則 $a = 1'b0$

以 1,1 來判斷：

➤ XOR: 相同為 0，不同為 1

➤ XNOR: 相同為 1，不同為 0

Valid Operators - Unary reduction operators (6)

□ 單元運算子 (Unary reduction operators)

➤ (6). $\sim^{\wedge}$ 或 $\wedge^{\sim}$, reduction XNOR 單元運算子

$\sim^{\wedge}$ 或 $\wedge^{\sim}$, reduction XNOR

用於偶同位產生器
(偶數個1)

b	0	1	00	01	10	11
$\sim^{\wedge}b$	1	1	1	0	0	1

真值表

若輸入資料的位元數為 1 個、則運算的結果為 1，相當於是設定的動作。

若輸入資料的位元數為 2 個 (含) 以上、且輸入資料值含有偶數個 1，則運算的結果為數 1，否則必為 0。

通常用於算術邏輯運算單元 (ALU) 中的 Even (偶同位元) 判斷旗標 (flags) 訊號。

reduction XNOR 和 reduction XOR 有互補的關係。

➤ 例：assign a = $\sim^{\wedge}b$;

若 b = 4'b0000, 則 a = 1'b1

若 b = 4'b0111, 則 a = 1'b0

若 b = 4'b1111, 則 a = 1'b1

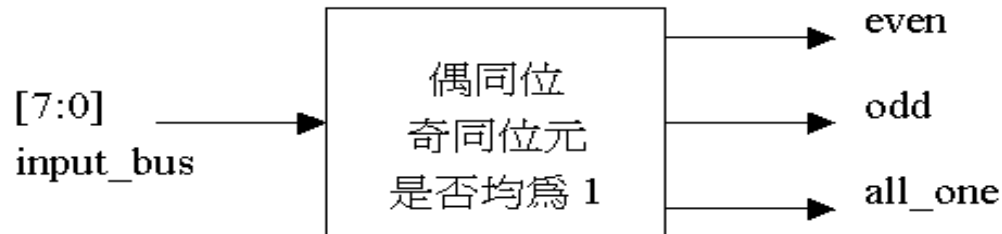
以 1,1 來判斷：

➤ XOR: 相同為 0，不同為 1

➤ XNOR: 相同為 1，不同為 0

Example for Unary reduction operators

- 範例: BitWise.V(單元運算子應用例子)，用以判別輸入的資料是否為奇同位、偶同位元及是否均為1。



// BitWise.V : 判別輸入的資料是否為奇同位、偶同位元及是否均為1。

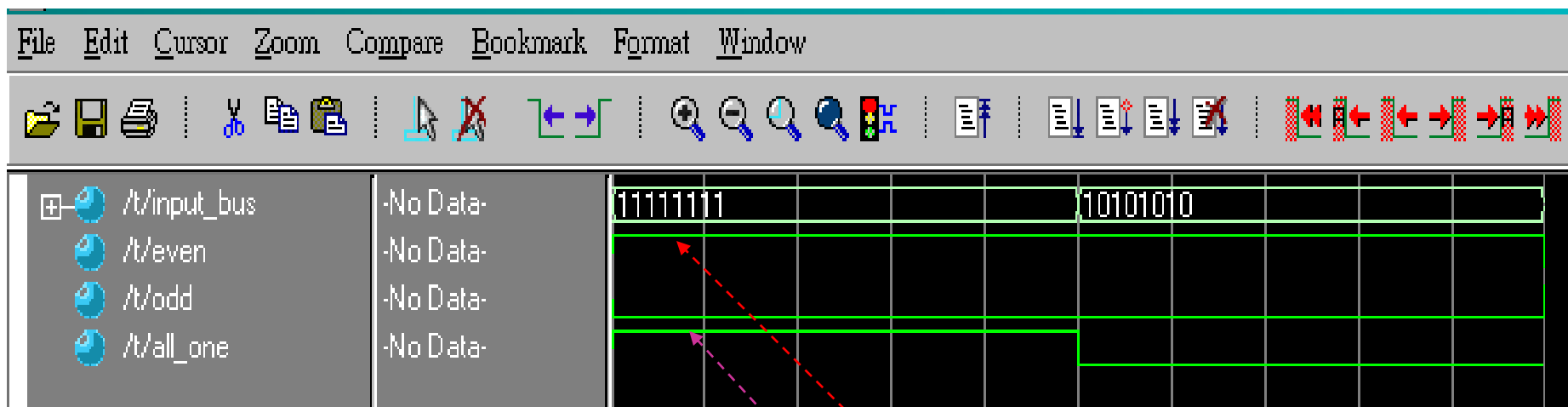
```
module BitWise(input_bus , even, odd, all_one);  
input  [7:0] input_bus;  
output even, odd, all_one;  
wire   even, odd, all_one;
```

XOR=相同為零

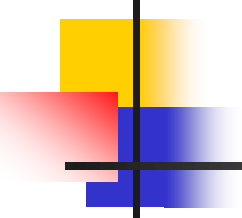
```
    assign odd = ^ input_bus;      // 奇同位的判別  
    assign even = ~^ input_bus;    // 偶同位的判別  
    assign all_one = & input_bus;  // 均為1的判別  
endmodule
```

Simulation of BITWISE

Core:1_03_BITWISE



```
assign odd = ^ input_bus; // 奇數個1則為1
assign even = ~^ input_bus; // 偶數個1則為1
assign all_one = & input_bus; // 均為1的判別
```

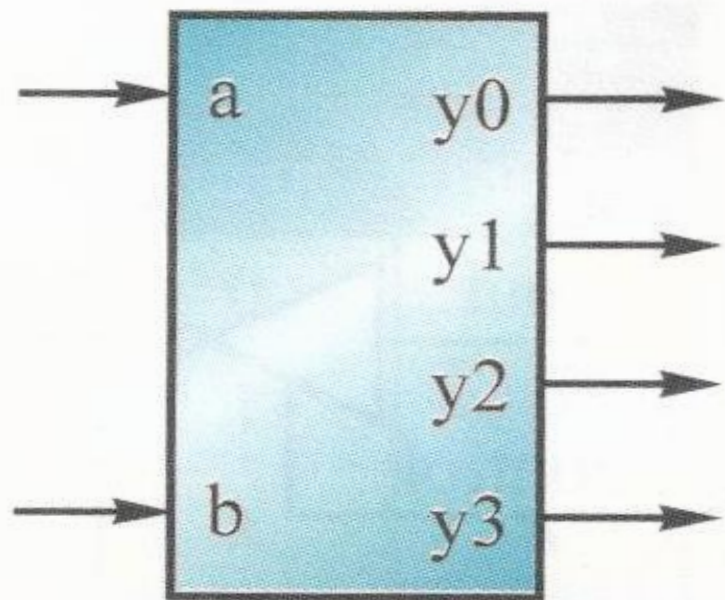


範例練習

2-4高態輸出解碼器

(3-002)

2-4解碼器方塊圖與真值表

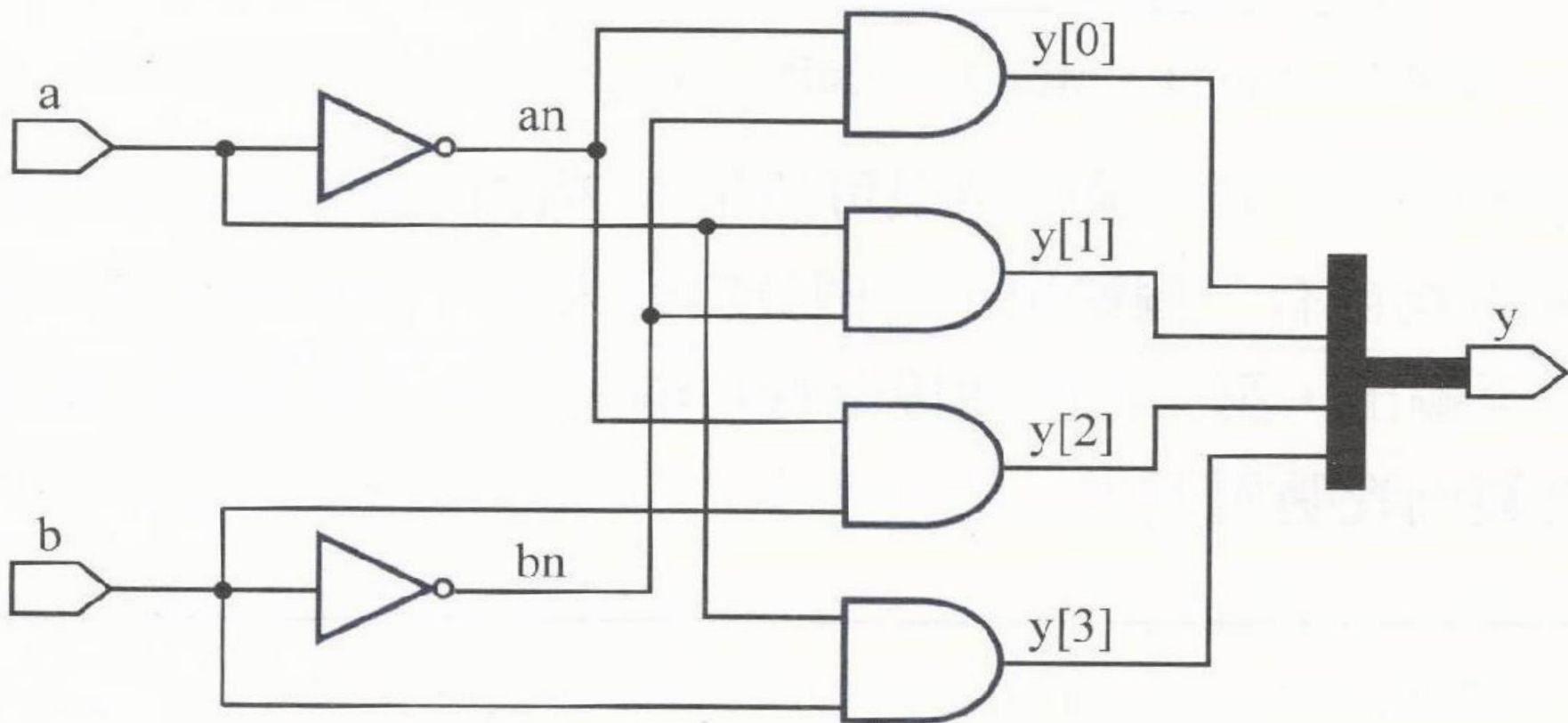


(a) 方塊圖

b	a	y3	y2	y1	y0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

(b) 真值表

2-4高態輸出解碼器



```
// -----
//      |      |      b a | y3 y2 y1 y0
// a---|      y0 |---  0 0 | 0  0  0  1
//      |      y1 |---  0 1 | 0  0  1  0
//      |      y2 |---  1 0 | 0  1  0  0
// b---|      y3 |---  1 1 | 1  0  0  0
//      |      |
// -----
//
// 2-4Decoder
// Filename: deco2_4g.v
//
module deco2_4g(a, b, y);

    input a, b;
    output[3:0] y;

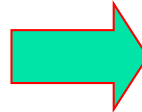
    wire an,bn;

    not(an, a);
    not(bn, b);

    and(y[0], an, bn);
    and(y[1], a, bn);
    and(y[2], an, b);
    and(y[3], a, b);

endmodule
```

Test bench



```
module deco2_4g_tb();
    reg a, b;
    wire [3:0] y;

    deco2_4g dec(.a(a), .b(b), .y(y));

    initial
        begin
            #0 a<=0;
            b<=0;
        end

    always
        begin
            #10 b<=0;
            a<=0;
            #10 b<=0;
            a<=1;
            #10 b<=1;
            a<=0;
            #10 b<=1;
            a<=1;
        end
    endmodule
```

Valid Operators - Logical operators (1)

- ❑ 邏輯運算子(logical operators)，包括有：(1). **!**, **Logical NOT**、(2). **&&**, **Logical AND**，以及 (3). **||**, **Logical OR** 等3種。
- ❑ 對於任何一個邏輯判斷而言：1為判斷條件成立(真)、0為判斷條件不成立(偽)。

➤ (1). **!**, **Logical NOT** 邏輯運算子(先OR再NOT)

!, Logical NOT

條件	經過 ! 後
不成立	成立
成立	不成立

a	0	1	00	01	10	11
!a	1	0	1	0	0	0

真值表

對於特定一筆輸入資料的任何一個位元：若第 n 個位置只要有一個為 1、則運算結果為 0，若第 n 個位置皆為 0、則運算結果為 1。

➤ 例：if (!a)

若 a = 4'b0000，則 !a = 1'b1、if (!a) 判斷成立。

若 a = 4'b0111，則 !a = 1'b0、if (!a) 判斷不成立。

若 a = 4'b1111，則 !a = 1'b0、if (!a) 判斷不成立。

Valid Operators - Logical operators (2)

❑ 邏輯運算子(logical operators)：

➤ (2). &&, Logical AND 邏輯運算子

&&, Logical AND

&&		條件 1	
		不成立	成立
條件 2	不成立	不成立	不成立
	成立	不成立	成立

對於二個判斷條件 (條件 1 及條件 2)：若這二個判斷條件皆成立、則結合判斷的結果為成立，若這二個判斷條件只要有一個是不成立、則結合判斷的結果為不成立。

➤ 例：if ((a > 5) && (b <= 2)) // 如果 a > 5 且 b <= 2

若 a = 3'b000 且 b = 2'b11, 則 if ((a > 5) && (b <= 2)) 判斷不成立

若 a = 3'b000 且 b = 2'b01, 則 if ((a > 5) && (b <= 2)) 判斷不成立

若 a = 3'b110 且 b = 2'b01, 則 if ((a > 5) && (b <= 2)) 判斷不成立

若 a = 3'b110 且 b = 2'b00, 則 if ((a > 5) && (b <= 2)) 判斷成立

Valid Operators - Logical operators (3)

❑ 邏輯運算子(logical operators)：

➤ (3). **||**, Logical OR 邏輯運算子

||, Logical OR

		條件 1	
		不成立	成立
條件 2	不成立	不成立	成立
	成立	成立	成立

對於二個判斷條件 (條件 1 及條件 2)：若這二個判斷條件皆不成立、則結合判斷的結果為不成立，若這二個判斷條件只要有一個是成立、則結合判斷的結果為成立。

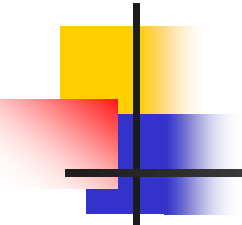
➤ 例：if ((a > 5) || (b <= 2)) // 如果 a > 5 或 b <= 2

若 a = 3'b000 且 b = 2'b11, 則 if ((a > 5) ~~&&~~ (b <= 2)) 判斷不成立

若 a = 3'b000 且 b = 2'b01, 則 if ((a > 5) ~~&&~~ (b <= 2)) 判斷成立

若 a = 3'b110 且 b = 2'b11, 則 if ((a > 5) ~~&&~~ (b <= 2)) 判斷成立

若 a = 3'b110 且 b = 2'b01, 則 if ((a > 5) ~~&&~~ (b <= 2)) 判斷成立



2's Complement Arithmetic

□ 2的補數算術 (Arithmetic)，包括有：(1) **+**, 加法運算子、(2) **-**, 減法運算子，以及 (3) *****, 乘法運算子 等3種。

➤ **+**, 加法運算子

例：`assign s = a + b;`

➤ **-**, 減法運算子

例：`assign s = a - b;`

➤ *****, 乘法運算子

例：`assign s = a * b;`

無”除法”與”餘數”等運算

Example for ADD or Subtract

```
// AddOrSub.V : 加減法 (Do Add or Subtract)
```

```
// 如果 op 等於 ADD, 則  $s = a + b$ 
```

```
// 如果 op 不等於 ADD, 則  $s = a - b$ 
```

```
module Add_or_Subtract(a, b, op, s);
```

```
parameter SIZE = 8;
```

```
parameter ADD = 1'b1;
```

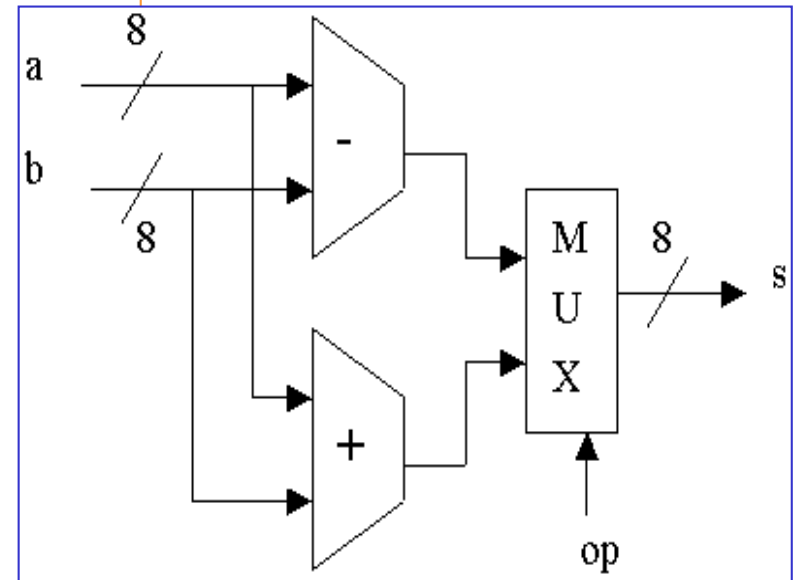
```
input op;
```

```
input [SIZE-1:0] a, b;
```

```
output [SIZE-1:0] s;
```

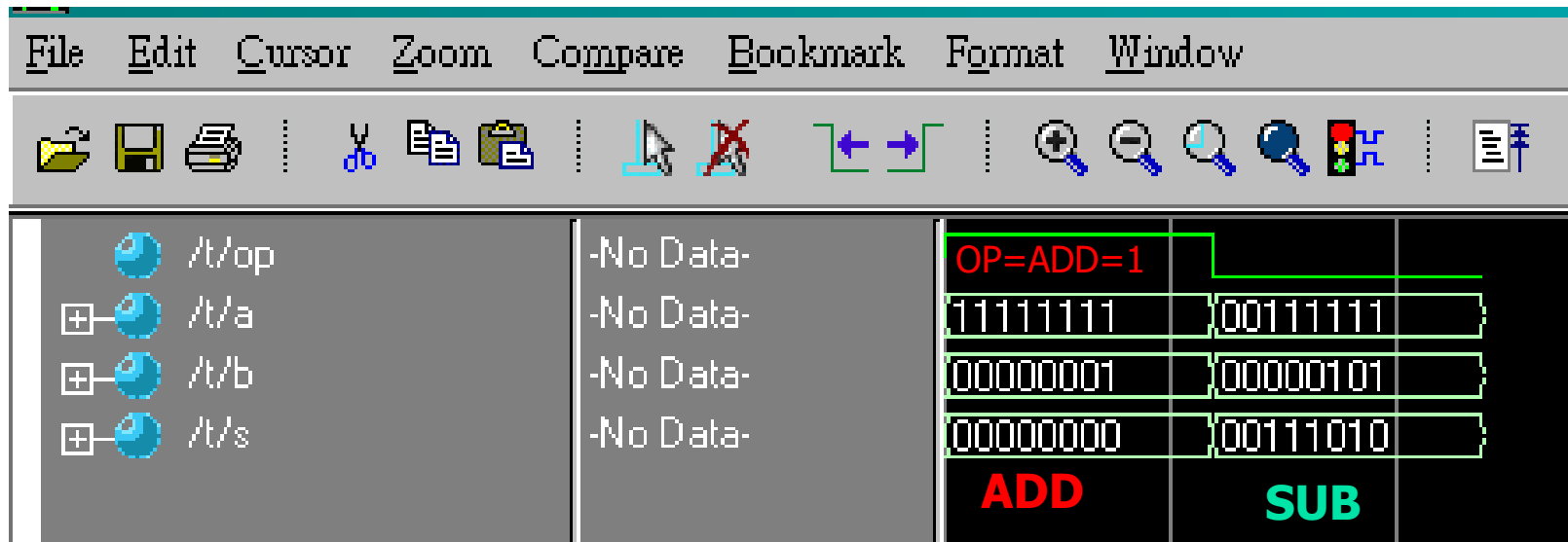
```
assign s = (op == ADD) ? a+b : a-b;
```

```
endmodule
```



Simulation of ADD_OR_SUB

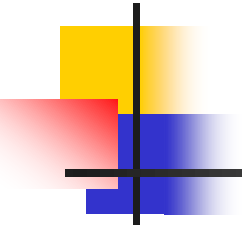
Core:1_05_ADDORSUB



Variable	Value
/t/op	-No Data-
/t/a	-No Data-
/t/b	-No Data-
/t/s	-No Data-

OP=ADD=1		
11111111	00111111	
00000001	00000101	
00000000	00111010	
ADD	SUB	

```
module Add_or_Subtract(a, b, op, s);  
parameter SIZE = 8;  
parameter ADD = 1'b1;  
input op;  
input [SIZE-1:0] a, b;  
output [SIZE-1:0] s;  
    assign s = (op == ADD) ? a+b : a-b;  
endmodule
```



Valid Operators - Relational operators

□ 關係運算子，包括有：(1). $>$, 大於運算子、(2). $<$, 小於運算子、(3). $\geq$, 大於等於運算子，以及 (4). $\leq$, 小於等於運算子 等4種。

➤ (1). $>$, 大於運算子

例：wire $a_bigger = a > b;$

➤ (2). $<$, 小於運算子

例：wire $a_less = a < b;$

➤ (3). $\geq$, 大於等於運算子

例：wire $a_gte = a \geq b;$

➤ (4). $\leq$, 小於等於運算子

例：wire $a_lte = a \leq b;$

Valid Operators - Equality operators (1)

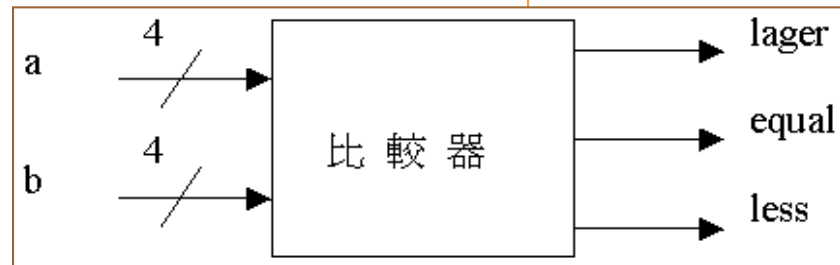
❑ 邏輯等式運算子，包括有：(1). ==, 相等運算子 (Logical equality)及 (2). !=, 不相等運算子 (Logical inequality) 等2種。

➤ (1). ==, 相等運算子 (Logical equality)

例1：比較器 (Comparator)

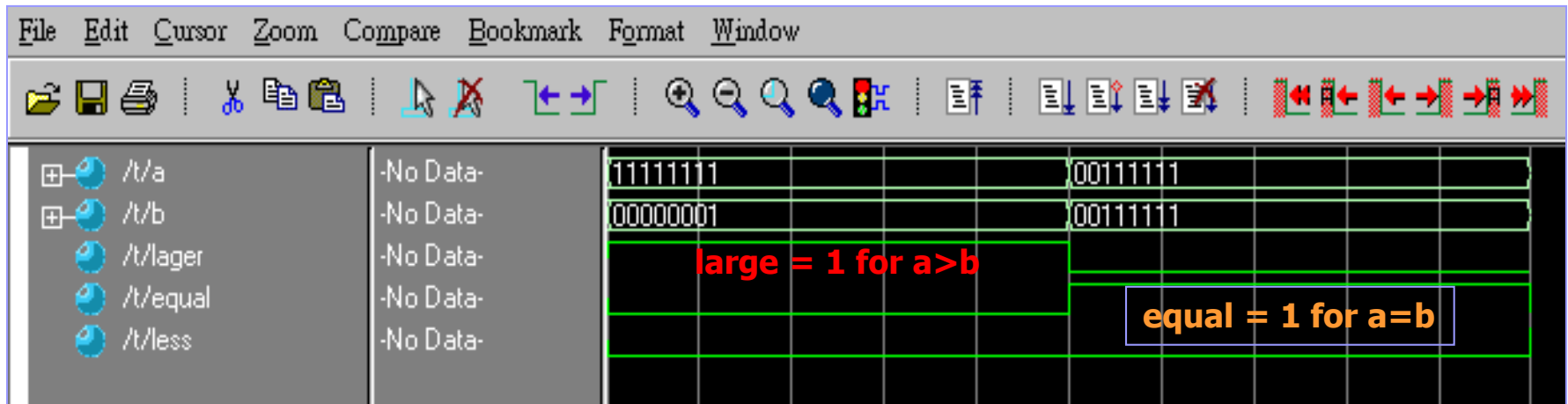
```
// Compare.V：比較器
module    Comparator (a, b, larger, equal, less);
parameter size = 8;
input     [size-1:0] a, b;
output    larger, equal, less;
wire      larger, equal, less;

assign larger = (a > b);
assign equal  = (a == b);
assign less   = (a < b);
endmodule
```



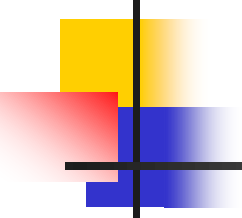
Simulation of Compare

Core:1_06_COMPARE



```
module    Comparator (a, b, larger, equal, less);
parameter size = 8;
input     [size-1:0] a, b;
output    larger, equal, less;
wire      larger, equal, less;

    assign larger = (a > b);
    assign equal = (a == b);
    assign less = (a < b);
endmodule
```



Valid Operators - Equality operators (2)

□ 等式運算子：

➤(2). !=, 不相等運算子 (Logical inequality)

以上的這些關係運算子都可用於向量式 (Vector)
的處理。

例：

```
input  [7:0] a, b;
wire   a_bigger, eq, a_less, not_eq;
wire   a_bigger = (a > b);
wire   eq = (a==b);
wire   a_less = (a < b);
wire   not_eq = (a!=b);
```


Valid Operators - Logical shift operators (1)

20231012

□ 移位運算子，包括有：(1). $\gg$, 右移運算子 (Right Shift)，以及(2). $\ll$, 左移運算子 (Left Shift) 等2種。

➤ (1). $\gg$, 右移運算子 (Right Shift)

例1：將 dividend 右移 amount 次, 每右移1次相當於是將 dividend 除以2。

// R_Shift.V : $\gg$, 右移運算子，範例

```
module R_Shift (dividend, amount, quotient);
```

```
input [7:0] dividend;
```

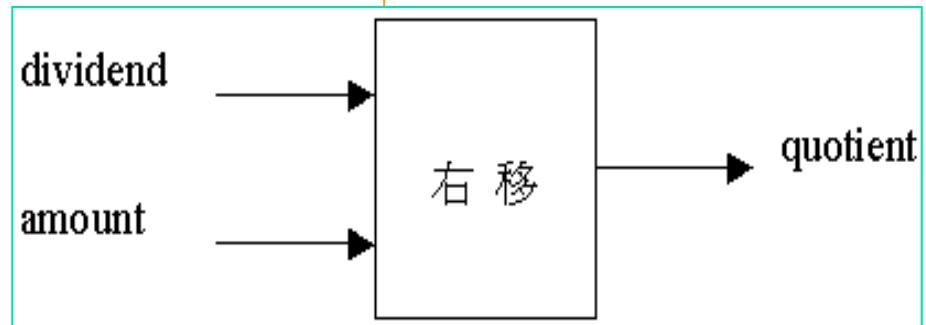
```
input [2:0] amount;
```

```
output [7:0] quotient;
```

```
wire [7:0] quotient;
```

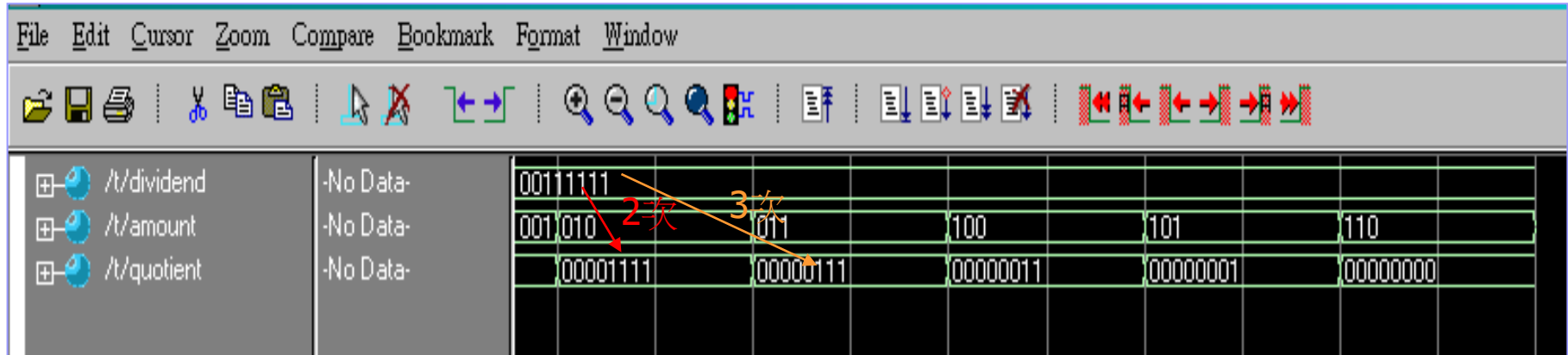
```
assign quotient = dividend  $\gg$  amount;
```

```
endmodule (quotient變數的值是dividend變數值右移amount次)
```



Simulation of R_Shift

Core:1_08_R_SHIFT



```
module R_Shift (dividend, amount, quotient);  
  input  [7:0] dividend;  
  input  [2:0] amount;  
  output [7:0] quotient;  
  wire   [7:0] quotient;  
  assign quotient = dividend >> amount;  
endmodule
```

Valid Operators - Logical shift operators (2)

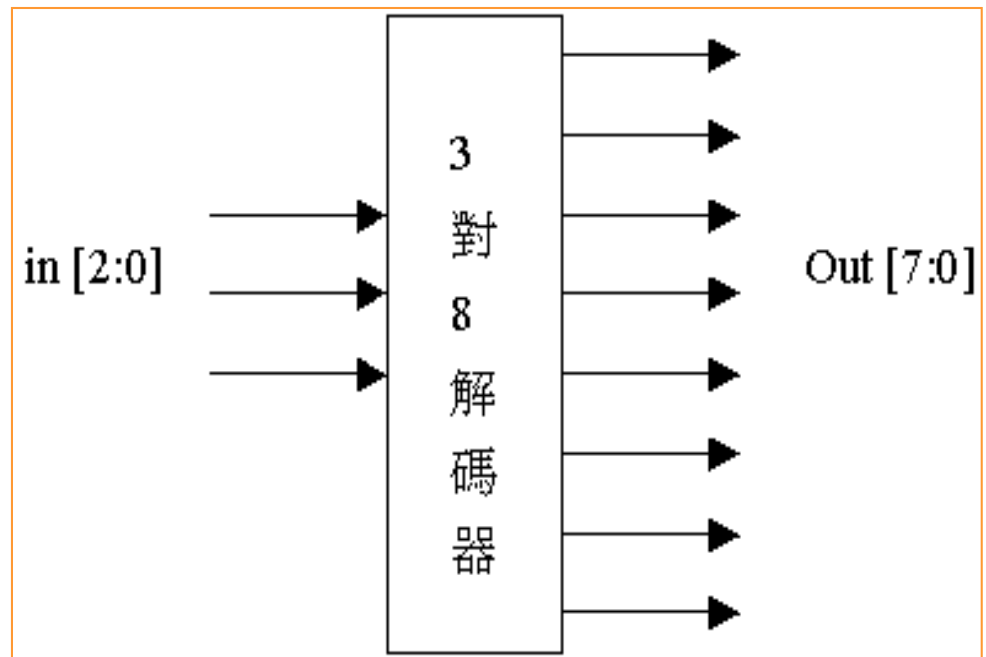
□ 移位運算子：

➤ (2). <<, 左移運算子 (Left Shift)

例2：解碼器 (Decoder)

```
//Decoder.V : 3對8的解碼器  
module Decoder(in, Out);  
input  [2:0] in;  
output [7:0] Out;  
wire  [7:0] Out;  
  
assign Out = 1'b1 << in;  
endmodule
```

(Out變數的值是數值1左移in次)



Simulation of Decoder

Core:1_09_DECODER

Signal	Value
/in	111
/out	10000000

Output 0	Output 1	Output 2	Output 3	Output 4	Output 5	Output 6	Output 7
000	001	010	011	100			
00000001	00000010	00000100	00001000	00010000			

```
module Decoder(in, Out);  
  input  [2:0] in;  
  output [7:0] Out;  
  wire  [7:0] Out;  
  assign Out = 1'b1 << in;  
endmodule
```

(Out變數的值是數值1左移in次)

Valid Operators - Conditional operators

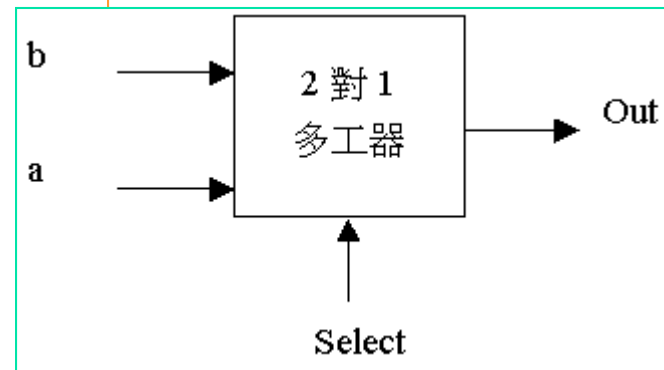
❑ ?: 條件運算子(Conditional operators)

➤ 表示法： $\text{Value} = \text{Test_Condition} ? \text{True_Part} : \text{False_Part};$

➤ 說明：如果 Test Condition 的值不為0的話，則 Value 的值將被設定為 True_Part 的值，否則 Value 的值將被設定為 False_Part 的值。

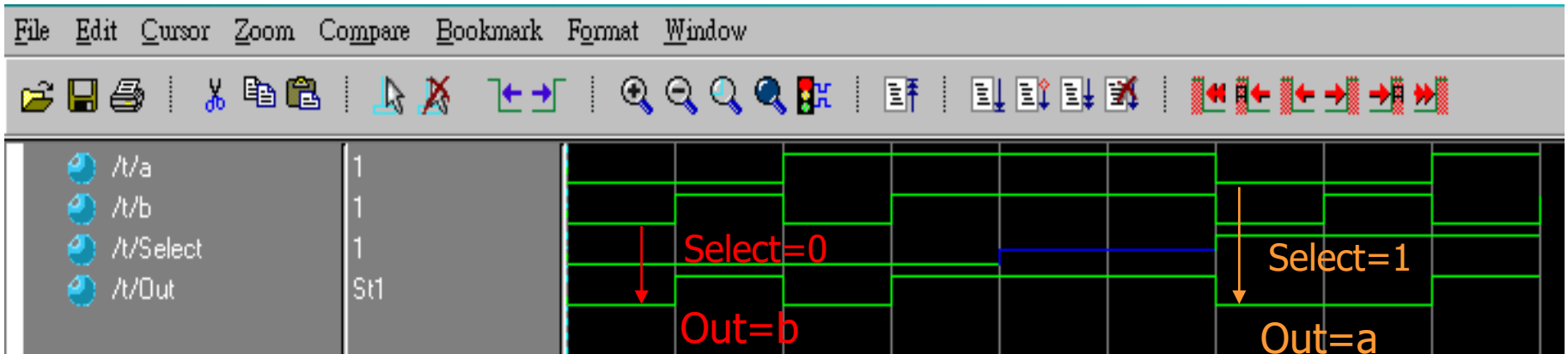
例1：2對1的多工器 (Multiplexier)

```
// Mux2to1.V：2對1的 Multiplexier
// 若 Select = 1, 則 Out = a, 否則 Out = b
module Mux2to1 (a, b, Select, Out);
input  a, b, Select;
output Out;
wire  Out;
      assign Out = Select ? a : b;
endmodule
```



Simulation of Mux2to1

Core:1_10_MUX2TO1



```
// 若 Select = 1, 則 Out = a, 否則 Out = b
module Mux2to1 (a, b, Select, Out);
input  a, b, Select;
output Out;
wire   Out;
    assign Out = Select ? a : b;
endmodule
```

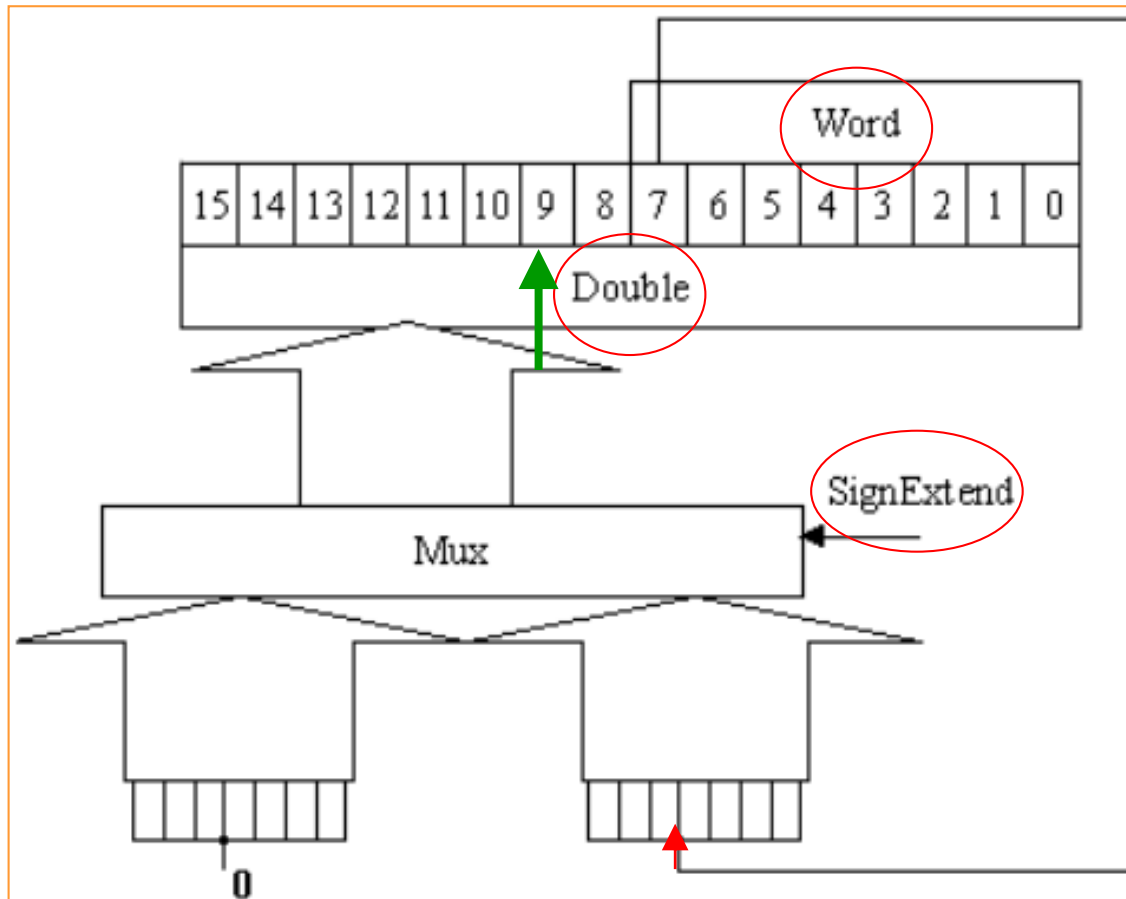
Valid Operators - Duplication operators (1)

□ {{}} 重覆 運算子(Duplication operators)

➤ BNF 表示法： $\{n\} \langle \text{expression} \rangle, \langle \text{expression} \rangle, \langle \dots \rangle \}$

➤ 說明：用以重覆產生 n 個 $\langle \text{expression} \rangle$ 。

➤ 例2：符號位元展開(Sign Extend)的電路



Valid Operators - Duplication operators (2)

例2：符號位元展開(Sign Extend)的Verilog 電路描述

// Sign_Ext.V

// 如果SignExtend = 1, 則 Double = 8 個 Word[7] + Word

// 否則 Double = 8 個 0 + Word

module Sign_Extend (SignExtend, Word, Double);

input SignExtend;

input [7:0] Word;

output [15:0] Double;

reg [15:0] Double;

always @(SignExtend or Word)

begin

if (SignExtend)

Double = {8{Word[7]}, Word}; // 重覆8個 Word[7] + Word

else (11000011=Word $\Rightarrow$ Double = 1111111111000011)

Double = {8'b0, Word}; // 重覆8個 0 + Word

end

(11000011=Word $\Rightarrow$ Double = 0000000011000011)

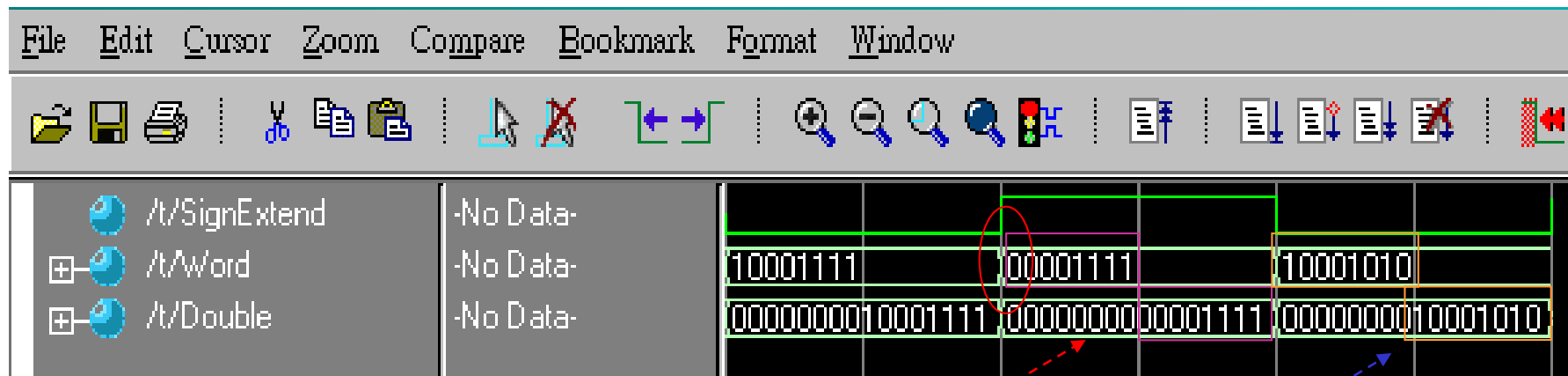
endmodule

(當**SignExtend**或**Word**的值有變動時，就開始執行下列指令)

(**Testbench** 會事先定義數值)

Simulation of Sign_ext

Core:1_12_SIGN_EXT



// 如果SignExtend = 1, 則 Double = 8 個 Word[7] + Word, 否則 Double = 8 個 0 + Word

```
module Sign_Extend (SignExtend, Word, Double);  
input  SignExtend;  
input  [7:0] Word;  
output [15:0] Double;  
reg    [15:0] Double;  
always @(SignExtend or Word)  
begin  
    if (SignExtend)  
        Double = {{8{Word[7]}}, Word}; // 重覆8個 Word[7] + Word  
    else  
        Double = {8'b0, Word}; // 重覆8個 0 + Word  
end  
endmodule
```

Valid Operators - Concatenation operators (1)

□ {} 連結運算子(Concatenation operators)

➤ BNF (Binary Normal Format)表示法：

<concatenation>

::= {<expression>, <expression>}

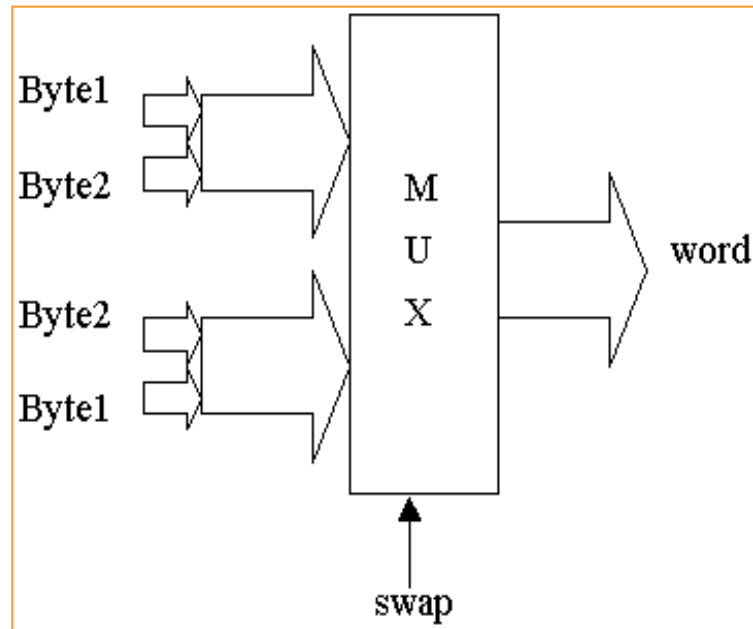
<multiple_concatenation>

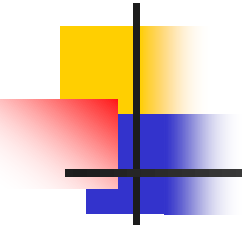
::= { <expression>, <expression>* }

多個

➤ 說明：用以連結二個或以上的值到一個變數中，或將一個變數的值劃分到二個或以上的變數中。

➤ 例2：連結 (Concatenate)

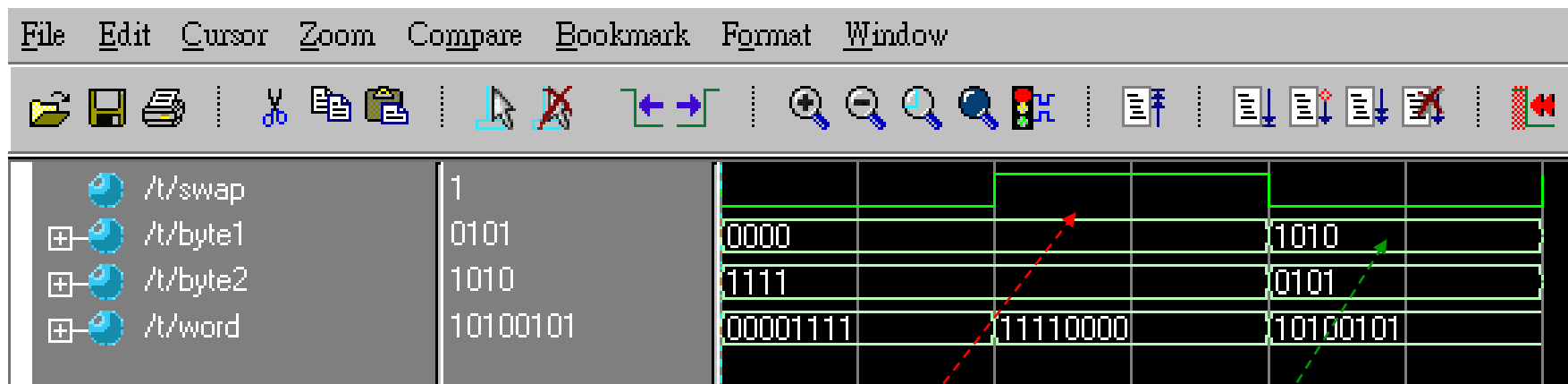




Valid Operators - Concatenation operators (2)

```
// Concat.V(連結)
// 如果swap = 1, 则 word = Byte2 連結(+) Byte1
//           否则 word = Byte1連結(+) Byte2
module  Concat (swap, byte1, byte2, word);
input   swap;
input   [3:0] byte1; (1000)
input   [3:0] byte2; (0001)
output  [7:0] word;
reg     [7:0] word;
  always @(swap or byte1 or byte2)
  begin
    if(swap)
      word = {byte2, byte1}; // word = byte2 << 4 + byte1;
    else
      (00011000)
      word = {byte1, byte2}; // word = byte1 << 4 + byte2;
  end
  (1000 0001)
endmodule
```

Core:1_13_CONCATE



```
// 如果 swap = 1, 則 word = Byte2 連結(+) Byte1, 否則 word = Byte1 連結(+) Byte2
module Concat (swap, byte1, byte2, word);
input swap;
input [3:0] byte1;
input [3:0] byte2;
output [7:0] word;
reg [7:0] word;
always @(swap or byte1 or byte2)
begin
    if (swap)
        word = {byte2, byte1}; // word = byte2 << 4 + byte1;
    else
        word = {byte1, byte2}; // word = byte1 << 4 + byte2;
    end
endmodule
```